CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G1
System Integration and Architecture

System Design Document (SDD)

Project Title: CampusGuru

Prepared By: Dabon, Kenn Xavier Cabonilas

Version: 1.0

Date: 02/28/2026
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1.0	02/21/26	Dabon, Kenn Xavier C.	Initial draft	Draft
1.0	02/26/26	Dabon, Kenn Xavier C.	Continued Initial Draft	Draft
1.0	02/26/26	Dabon, Kenn Xavier C.	Updated MoSCoW	Draft
1.0	02/27/26	Dabon, Kenn Xavier C.	Worked on Chapters 5.0 - 8.0	Draft
1.0	02/28/26	Dabon, Kenn Xavier C.	Final touches	**Final**

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4

1.0 INTRODUCTION 5

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5

3.0 NON-FUNCTIONAL REQUIREMENTS 8

4.0 SYSTEM ARCHITECTURE 9

5.0 API CONTRACT & COMMUNICATION 10

Authentication Endpoints 11

User Registration 11

User Login 11

6.0 DATABASE DESIGN 13

7.0 UI/UX DESIGN 14

8.0 PLAN 17

EXECUTIVE SUMMARY

1.1 Project Overview

CampusGuru is a campus-exclusive platform that handles the logistics of having to meet up with a tutor on campus. It allows students to sign-up as “Tutors” to share their expertise while “Learners” can discover, book, and review these peer tutors through the mobile app. The system includes a Spring Boot backend API, React web application, and Kotlin Android mobile app, all integrated to provide a seamless experience in both searching for a tutor and sharing knowledge.

1.2 Objectives

1. Develop a campus-exclusive peer-to-peer tutoring marketplace with restricted university-email authentication to ensure a secure and trusted environment.
2. Implement a three-tier architecture using Spring Boot (backend), React (web), and Android (mobile)
3. Design a responsive user interface that works consistently across web and mobile platforms
4. Create a robust listing and booking system that allows tutors to manage availability and enables learners to discover, book, and review peer expertise.
5. Design and integrate an administrative verification workflow for reviewing "Proof of Competence" documents and moderating content to maintain academic integrity.
6. Establish a standardized feedback mechanism through a 1–5 star rating and review system to ensure quality and accountability within the campus community.
7. Deploy a centralized tutor dashboard for managing pending requests, scheduling meeting links, and displaying specialized subject competence.

1.3 Scope

Included Features:

- **Tutors can:**
  - Create "listings" where they...
    - Specify the subject they will teach.
    - Specify their available time for booking (e.g. Monday, Wednesday, and Friday at 3:00 PM until 5:00 PM)
    - Specify an asking price for their services. (Can be free as well)
    - Specify preferred payment method (Cash, GCash, etc.)
    - Add contact information (MS Teams or other Social Media)
    - Upload some form of proof of competence (transcript or grade for that specific subject).
  - Handle booking requests
    - The Dashboard includes a table showing "Pending Requests" showing the student's name, subject, and time for the specific booking.
    - The Tutor can either accept or decline the request.
    - If the request is ACCEPTED, the Tutor is prompted by a modal to paste a meeting link where the Learner and Tutor can communicate (Zoom, Google Meet, MS Teams, etc.).
    - If the request is DECLINED, the Tutor is prompted by a modal to enter the reason as to why the request is denied.
  - Book other tutors if they wish.
- **Learners can:**
  - On the home page:
    - View recently posted listings.
    - Filter by Subject, Rating, and Price Range
    - Rating (e.g. show only 4+ star tutors)
  - When the Learner clicks on a listing, they can see the...

        - The details entered by the specific Tutor that created the listing (Subject, Available time, Contact Info, Proof of Competence)
        - The Tutor´s profile (Bio and Rating)
        - Field that specifies payment type (Cash, GCash, etc.)
        - Book this tutor button
    - After the session has ended, the learner will be able to…
        - Rate the Tutor 1-5 Stars with an optional text comment.

- **Admins can:**
    - Verify the Credentials of Tutors
        - Can: View the uploaded "Proof of Competence" (transcript/grades) in a "View-Only" Modal.
        - Can: Approve or Reject a Tutor application.
        - Can: Review the Tutor's asking price to ensure it is reasonable and within campus guidelines.
        - Can: Select a rejection reason from a predefined list (e.g., "Document Blurred," "Grade below threshold," "Wrong subject", “Unreasonable Price”).
    - Moderate the uploaded content
        - Can: Delete or Hide specific listings that violate guidelines (e.g., "I will do your homework for you" – which is academic dishonesty, versus "I will teach you").
        - Can: Remove reviews that contain hate speech or do not reflect a genuine service interaction.

Excluded Features:

- Video Call / Messaging
- Automated Session Tracking
- Cross Institutional Support
- Calendar Synchronization

- The system will not process actual credit card or digital wallet transactions (e.g., Stripe, PayPal integration).

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the Marketplace MVP system. It provides detailed requirements, architectural decisions, API contracts, database design, and implementation roadmap to guide development and ensure all components integrate seamlessly.

This document serves as the comprehensive design specification for CampusGuru, a cross-platform peer-to-peer tutoring marketplace. The primary intent is to serve as the definitive technical blueprint for the Minimum Viable Product (MVP), ensuring that the three primary system components—the React Web Application, the Kotlin Android App, and the Spring Boot REST API—function as a unified ecosystem.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: CampusGuru
Domain: Education/Peer-to-peer tutoring
Primary Users:

1. Students/Learners
2. Students who wish to teach

Problem Statement: Students often struggle with finding tutors with subjects they struggle with, finding it too expensive or daunting. While there are many students who excel and are capable of teaching these subjects, there is no centralized platform to connect those in need with those who can help. Existing ideas such as group chats or bulletin boards are disorganized and often lack scheduling tools.

Solution: A campus-exclusive marketplace that makes learning accessible for everyone. CampusGuru handles the logistics part of the meeting. It allows students to sign-up as “Tutors” to share their expertise while “Learners” can discover, book, and review these peer tutors through the web or mobile app.

2.2 Core User Journeys

Journey 1: Learner Discovery & Booking

1. The student signs up using university email (e.g juan.delacruz@cit.edu) via Web or Mobile.
2. The student browses the home feed for recently posted tutor listings.
3. The student uses the filter feature (by subject, rating, price range) to find a specific match.
4. The student clicks a listing to view the tutor's profile, "Proof of Competence," and price.
5. Selects an available time and payment method and clicks "Book this tutor."
6. Receives a meeting link once the Tutor accepts the request.

Journey 2: Tutor Listing & Management

1. The student registers as a Tutor, providing the necessary information (program, specialized subject, price)
2. Uploads a transcript or grade screenshot as proof of expertise
3. Sets availability and contact info
4. Views "Pending Requests" on the dashboard
5. Accepts a request and provides a virtual meeting link

Journey 3: Administrator Oversight

1. Admin logs into the admin portal to view pending Tutor applications.
2. Inspects the uploaded "Proof of Competence" in a **view-only** modal.
3. Approves the tutor or rejects them with a predefined reason (e.g. "Unreasonable Pricing")

2.3 Feature List

MUST HAVE

1. University email authentication
2. Tutor listing creation (subject, price, availability)
3. Learners can book tutors from listings.
4. Document upload (Proof of Competence)
5. Booking request system
6. Rating tutors
7. Admin portal where listing applications are accepted/rejected (with reason)
8. Cross platform sync (Web & Mobile)

9. User Profiles (Bio)

SHOULD HAVE

1. Filtering by subjects, rating, and price range.

COULD HAVE

1. “Save/Favorite” tutor listings
2. Dark mode UI

WON'T HAVE

1. Real payment processing
2. In-app video calling
3. In-app messaging
4. Automated calendar syncing

2.4 Detailed Feature Specifications

Feature: University-Exclusive Authentication

- **Screens:** Registration, Login, Profile Setup.
- **Fields:** Email, Password, Firstname, Lastname, Role.
- **Validation:** Regex restriction to institutional domains (@university.edu); Password hashing with **BCrypt** (12 salt rounds).
- **Integration:** Stateless session management using **JWT (JSON Web Tokens)** stored securely in the web browser and mobile device.
- **API Endpoints:** POST /auth/register, POST /auth/login.

Feature: Peer Tutor Listings

- **Screens:** Create Listing (Tutor), Home Page (Learner), Listing Details.
- **Display:** Grid layout featuring Tutor name, average rating, subject, and asking price.
- **Tutor Logic:** Tutors must upload a "Proof of Competence" URL (linked to cloud storage) before a listing is submitted for admin review.
- **Search/Filter:** Learners can fetch listings and filter results by subject or rating.
- **API Endpoints:** POST /listings (Create), GET /listings (Fetch/Search).

Feature: Peer-to-Peer Booking

- **Screens:** Booking Request Form, Request Dashboard (Tutor), Booking Status (Learner).
- **Process: Request:** Learner selects a time and sends a message to the Tutor.

    - **Approval:** Tutor accepts the request and *must* provide a virtual meeting link (Zoom, MS Teams, etc.).
    - **Rejection:** Tutor declines the request and *must* provide a rejection reason.
- **API Endpoints:** POST /bookings, PUT /bookings/{bookingId}/status.

Feature: Administrative Verification & Moderation

- **Screens:** Admin Pending Queue, Document Verification Modal.
- **Verification Logic:** * Admins review the proofOfCompetenceUrl in a view-only modal.
    - Admins change listing status to APPROVED or REJECTED.
    - Rejected listings require a reason chosen from a predefined list.
- **Access Control:** All endpoints require an ADMIN role claim within the JWT.
- **API Endpoints:** GET /admin/listings/pending, PUT /admin/listings/{listingId}/verify.

Feature: Marketplace Dashboard

- **Screens:** Active Bookings/Dashboard.
- **Logic:** Users select the "View Listing" button to view important details about the listing and to actually book the tutor (with the Book Tutor button).
- **Data Collected:** Listing ID, Requested Time, and Learner Message.
- **Process:** Validate user role -> Create booking entry -> Update Tutor's pending request table.
- **API Endpoints:** POST /bookings, GET /bookings/{id} (to view specific status).

2.5 Acceptance Criteria

AC-1: Institutional Access Control

- ➢ Given I attempt to register with an email that is not an institutional email (e.g. gmail.com)
- ➢ When I click "Sign Up"
- ➢ Then the system should reject the request with an error message: "Only university email addresses are permitted."

AC-2: Proof of Competence workflow

- ➢ Given a Tutor has uploaded their transcript and submitted a listing.
- ➢ When an Admin reviews the document and clicks "Approve."

- ➢ Then the listing should become visible to all Learners on the Home Page.

AC-3: Session Feedback Loop

- ➢ Given a booking request has been "Accepted" and the time has passed.
- ➢ When the Learner submits a 5-star rating and comment.
- ➢ Then the Tutor's average rating should update in real-time across both Web and Mobile platforms.

AC-4: Tutor Availability Management

- ➢ Given I am logged in as a Tutor.
- ➢ When I create a listing and specify my available days (e.g., Tuesday/Thursday) and time slots.
- ➢ Then those specific slots should be the only options selectable by a Learner on the booking page.
- ➢ And the system must prevent me from saving a list without at least one time slot or a valid price.

AC-5: The Booking Request Lifecycle (Rejection)

- ➢ Given a Learner has submitted a booking request to a Tutor.
- ➢ When the Tutor clicks “Decline”.
- ➢ Then a modal must appear requiring the Tutor to provide a reason for the rejection.
- ➢ And once submitted, the Learner should see the status update to “Declined”on their dashboard along with the Tutor’s reason.

AC-6: The Booking Request Lifecycle (Acceptance)

- ➢ Given a Tutor receives a “Pending Request”r.
- ➢ When the Tutor selects “Accept”.
- ➢ Then the Tutor must be prompted to enter a valid scheduled meeting URL (e.g., Zoom/Google Meet).
- ➢ And upon saving, the status must change to "Accepted," and the meeting link should become clickable/visible only to the Learner who made the booking.

AC-7: Learner requesting a Tutor

- ➢ Given I am a learner who is requesting a Tutor.
- ➢ When I select “Request”.
- ➢ Then I am first prompted to input valid contact information in case of cancellation or future possible conflicts.

AC-8: Tutor inputs an unreasonable asking price

- ➢ Given I am a Tutor who is finalizing my listing application.
- ➢ When I input an unreasonable asking price (e.g. 10,000 php) and proceed.

- ➢ Then the admin (from the Admin portal) will deny my listing application with a message stating the reason why the application was declined.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance and Integration Requirements

- **Data Synchronization:** Updates made on the Mobile app must reflect on the React Web portal within 2 seconds or less.
- **API Response Time:** POST and PUT requests (e.g., submitting a booking or verifying a tutor) must complete in 2 seconds or less.
- **Concurrency:** The system must support at least 30 concurrent users per campus without degradation in database performance (to be tested using Apache JMeter)
- **Media Retrieval:** Images of "Proof of Competence" (transcripts) must load in Admin view-only modal within 3 seconds or less.

3.2 Security Requirements

- **Domain Restriction**: The system must strictly reject any authentication attempt that does not originate from the *@university.edu*.
- **Identity Management:** Use JWT (JSON Web Tokens) with a 24-hour expiration for stateless session management across platforms.
- **Encryption:** All sensitive data (passwords) must be hashed using BCrypt.
- **Role-Based Access Control (RBAC):** Admin-only endpoints must return a 403 Forbidden status if accessed by a user with a "Student" or "Tutor" role.

3.3 Compatibility Requirements

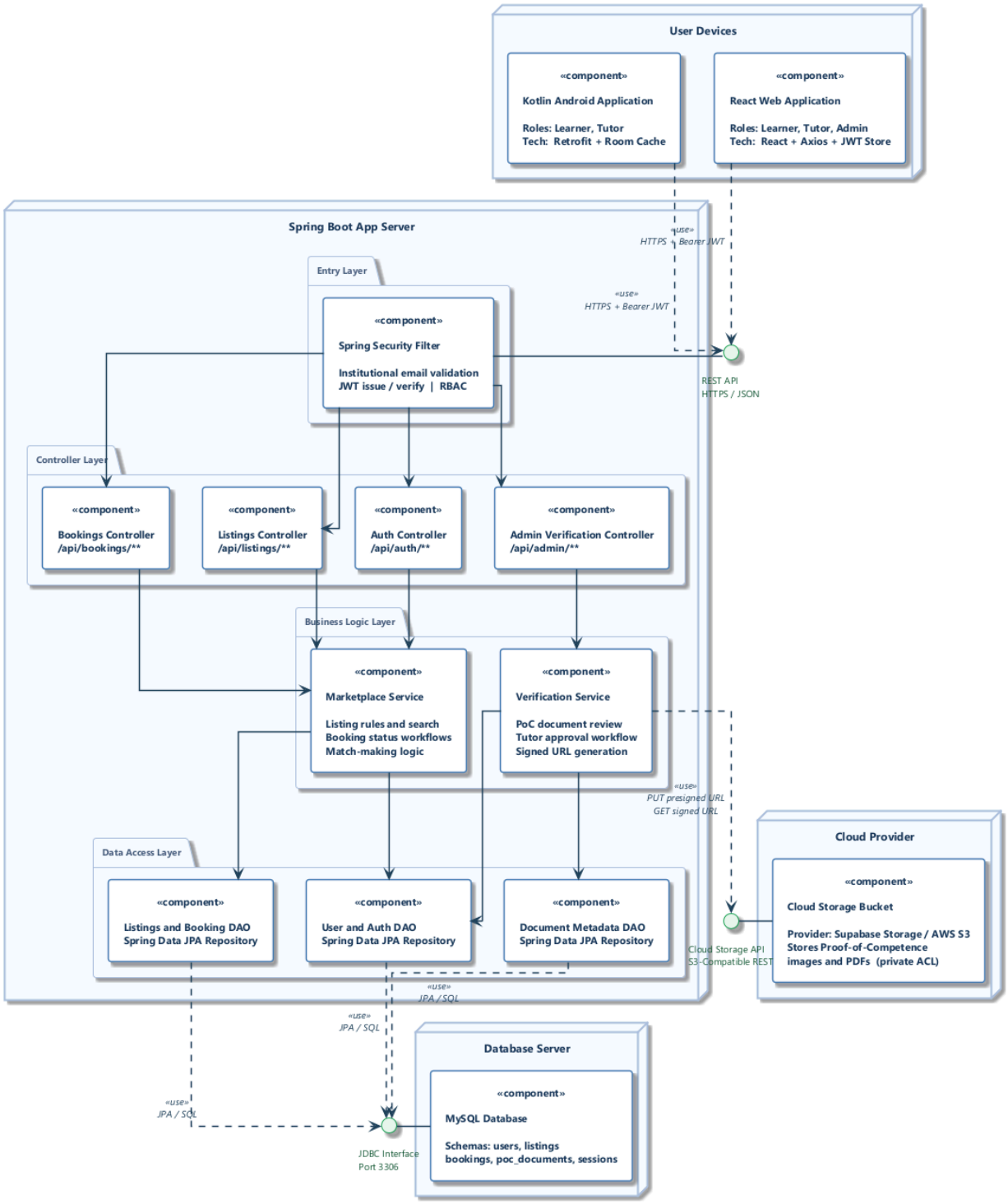
- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** API Level 24+ (Android 7.0+)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+

3.4 Scalability Requirements

- **Statelessness:** The Spring Boot backend must remain stateless to allow for horizontal scaling as the student user base grows.
- **Database Normalization:** The schema must be optimized (3rd Normal Form) to prevent redundant data storage as thousands of tutoring records are created.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 21, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** MySQL 8.4
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, Jetpack Compose, Retrofit, Room
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Render, APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header

Response Structure	{
	"success": boolean,
	"data": object|null,
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	{ “email”: <email>, “password”: <password>,

	“first_name”: <firstname>, “last_name”: <lastname>, “role”: <role> }

Response Structure	{
	"success": boolean,
	"data": {
	user {
	“email”: <email>,
	“first_name”: <firstname>,
	“last_name”: <lastname>,
	“role”: <role>
	},
	“access_token”: <token>,
	“refresh_token”: <token>,
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	JWT

Request Payload	{
	“identifier”: <identifier>,
	“password”: <password>,
	}

Response Structure	{
	"success": boolean,
	"data": {
	“user”: {
	“email”: <email>,
	“first_name”: <firstname>,
	“last_name”: <lastname>,
	“role”: <role>,
	},
	“access_token”: <token>,

	“refresh_token”: <token>,
	},
	"error": {
	"code": string,
	"message": string
	},
	
	"timestamp": string
	}

Listings Endpoints

Create a Listing

Description	Create a Listing
API URL	/listings
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	JWT

Request Payload	{
	"subject": <subject>,
	“description”: <description>,
	“asking_price” : <asking_price>
	"availability_slots": <availability_slots>,
	"askingPrice": <askingPrice>,
	"proofOfCompetenceUrl":
	<proofOfCompetenceUrl>,
	“status” : <status>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"listing_id": <listingId>,
	"status": <status>,
	"createdAt": <createdAt>
	},
	"error": {
	"code": string,
	"message": string
	},
	"timestamp": string
	}

Get Listings

Description	Fetch all of the Listings
API URL	/listings
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	JWT
Request Payload	-

Response Structure	{
	"success": boolean,
	"data": {
	"listings": [
	{
	"listing_id": <listing_id>,
	"tutor": {
	"tutor_id": <tutor_id>,
	"first_name": <first_name>,
	"last_name": <last_name>,
	"average_rating": <average_rating>
	},
	"subject": <subject>,
	"availability_slots": <availability_slots>,
	"asking_price": <asking_price>
	}
	]
	},
	"error": {
	"code": string,
	"message": string
	},
	"timestamp": string
	}

Booking Endpoints

Create Booking Request

Description	Create a Tutor Booking Request
API URL	/bookings
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	JWT
Request Payload	{

	"listing_id": <listing_id>, "requested_time": <requested_time>, "message": <message> }

Response Structure	{
	"success": boolean,
	"data": {
	"booking": {
	"booking_id": <booking_id>,
	"status": <status>,
	"requested_time": <requested_time>
	}
	},
	"error": {
	"code": string,
	"message": string
	},
	"timestamp": string
	}

Update Booking Status

Description	Update booking status
API URL	/bookings/{bookingId}/status
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	JWT

Request Payload	{
	"status": <status>,
	"meeting_link": <meeting_link>,
	"decline_reason": <decline_reason>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"booking": {
	"booking_id": <booking_id>,
	"status": <status>,
	"meeting_link": <meeting_link>,
	"updated_at": <updated_at>
	}
	},
	"error": {
	"code": string,

	"message": string }, "timestamp": string }

Review Endpoints

Submit a Review

Description	Submit a Review for a completed session
API URL	/reviews
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	JWT

Request Payload	{
	"booking_id": <booking_id>,
	"rating": <rating>,
	"comment": <comment>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"review": {
	"review_id": <review_id>,
	"booking_id": <booking_id>,
	"tutor_profile_id": <tutor_profile_id>,
	"rating": <rating>,
	"comment": <comment>,
	"created_at": <created_at>
	}
	},
	"error": {
	"code": string,
	"message": string
	},
	"timestamp": string
	}

Get Reviews for a Tutor

Description	Retrieve all reviews associated with a specific tutor profile
API URL	/reviews/tutor/{tutor_profile_id}
HTTP Method	GET

Format	JSON for all requests/responses
Authentication	JWT
Request Payload	-

Response Structure	{
	"success": boolean,
	"data": {
	"reviews": [
	{
	"review_id": <review_id>,
	"booking_id": <booking_id>,
	"reviewer": {
	"user_id": <user_id>,
	"first_name": <first_name>,
	"last_name": <last_name>
	},
	"rating": <rating>,
	"comment": <comment>,
	"created_at": <created_at>
	}
	],
	"average_rating": <average_rating>,
	"total_reviews": <total_reviews>
	},
	"error": {
	"code": string,
	"message": string
	},
	"timestamp": string
	}

Tutor Profile Endpoints

Get Reviews for a Tutor

Description	Retrieve the public profile of a specific tutor
API URL	/tutor/profile/{tutor_profile_id}
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	JWT
Request Payload	-

Response Structure	{
	"success": boolean,
	"data": {
	"profile": {
	"tutor_profile_id": <tutor_profile_id>,
	"user": {
	"user_id": <user_id>,
	"first_name": <first_name>,
	"last_name": <last_name>
	},
	"bio": <bio>,
	"average_rating": <average_rating>,
	"is_verified": <is_verified>,
	"profile_picture_url":
	<profile_picture_url>
	}
	},
	"error": {
	"code": string,
	"message": string
	},
	"timestamp": string
	}

Fetch User Tutor Profile

Description	Retrieve the currently authenticated tutor's own profile
API URL	/tutor/profile/me
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	JWT
Request Payload	-

Response Structure	{
	"success": boolean,
	"data": {
	"profile": {
	"tutor_profile_id": <tutor_profile_id>,
	"user": {
	"user_id": <user_id>,
	"first_name": <first_name>,
	"last_name": <last_name>,
	"institutional_email":
	<institutional_email>
	},

	"bio": <bio>,
	"average_rating": <average_rating>,
	"is_verified": <is_verified>,
	"profile_picture_url": <profile_picture_url>
	}
	},
	"error": {
	"code": string,
	"message": string
	},
	"timestamp": string
	}

Create Tutor Profile

Description	Create a tutor profile for the currently authenticated user
API URL	/tutor/profile/me
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	JWT

Request Payload	{
	"bio": <bio>,
	"profile_picture_url": <profile_picture_url>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"profile": {
	"tutor_profile_id": <tutor_profile_id>,
	"user_id": <user_id>,
	"bio": <bio>,
	"average_rating": 0,
	"is_verified": false,
	"profile_picture_url": <profile_picture_url>,
	"created_at": <created_at>
	}
	},
	"error": {
	"code": string,
	"message": string
	},
	"timestamp": string

	}

UpdateTutor Profile

Description	Update the bio or profile picture of the authenticated tutor's profile
API URL	/tutor/profile/me
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	JWT

Request Payload	{
	"bio": <bio>,
	"profile_picture_url": <profile_picture_url>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"profile": {
	"tutor_profile_id": <tutor_profile_id>,
	"bio": <bio>,
	"profile_picture_url": <profile_picture_url>,
	"updated_at": <updated_at>
	}
	},
	"error": {
	"code": string,
	"message": string
	},
	"timestamp": string
	}

Admin Endpoints

Get Pending Listings

Description	Create a Listing
API URL	/admin/listings/pending
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	JWT
Request Payload	-

Response Structure	{
	"success": boolean,
	"data": {
	"pendingListings": [
	{
	"listing_id": <listing_id>,
	"tutor": {
	"tutor_id": <tutor_id>,
	"first_name": <first_name>,
	"last_name": <last_name>
	},
	"subject": <subject>,
	"asking_price": <asking_price>,
	"proof_of_competence_url": <proof_of_competence_url>,
	"submitted_at": <submitted_at>
	}
	]
	},
	"error": {
	"code": string,
	"message": string
	},
	"timestamp": string
	}

Verify Listing

Description	Create a Listing
API URL	/admin/listings/{listingId}/verify
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	JWT

Request Payload	{
	"status": <status>,
	"rejection_reason": <rejection_reason>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"listing": {
	"listing_id": <listing_id>,
	"status": <status>,
	"rejection_reason": <rejection_reason>,

	"verified_at": <verified_at>,
	"verified_by": <admin_id>
	}
	},
	"error": {
	"code": string,
	"message": string
	},
	"timestamp": string
	}

Get Verification History for a Listing

Description	Retrieve the full audit log of all verification actions taken on a specific listing
API URL	/admin/listings/{listing_id}/verifications
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	JWT
Request Payload	-

Response Structure	{
	"success": boolean,
	"data": {
	"listing_id": <listing_id>,
	"verifications": [
	{
	"verification_id": <verification_id>,
	"admin": {
	"user_id": <user_id>,
	"first_name": <first_name>,
	"last_name": <last_name>
	},
	"decision": <decision>,
	"rejection_reason": <rejection_reason>,
	"verified_at": <verified_at>,
	"created_at": <created_at>
	}
	],
	"total_actions": <total_actions>
	},
	"error": {
	"code": string,
	"message": string

	}, "timestamp": string }

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1	{
	"success": false,
	"data": null,
	"error": {
	"code": "AUTH-004",
	"message": "Domain restricted",
	"details": "Only @university.edu email addresses are permitted for registration."
	},
	"timestamp": "2026-02-27T10:30:00Z"
	}

Example 2	{
	"success": false,
	"data": null,
	"error": {
	"code": "BOOKING-002",
	"message": "Validation failed on status update",
	"details": {
	"meetingLink": "A valid meeting link is required when accepting a booking.",
	"declineReason": "Must be null when status is ACCEPTED."
	}
	},
	"timestamp": "2026-02-27T10:35:00Z"

	}

Common Error Codes

Authentication & Authorization

- **AUTH-001:** Invalid credentials (wrong password or email)
- **AUTH-002:** Token expired or malformed
- **AUTH-003:** Insufficient permissions (role mismatch)
- **AUTH-004:** Non-institutional email domain detected

Validation & Data

- **VALID-001:** Payload validation failed (e.g., missing fields, weak password)
- **DB-001:** Resource not found (e.g., invalid listing_id)
- **DB-002:** Duplicate entry (e.g., user already applied as a Tutor)

Marketplace & Business Logic

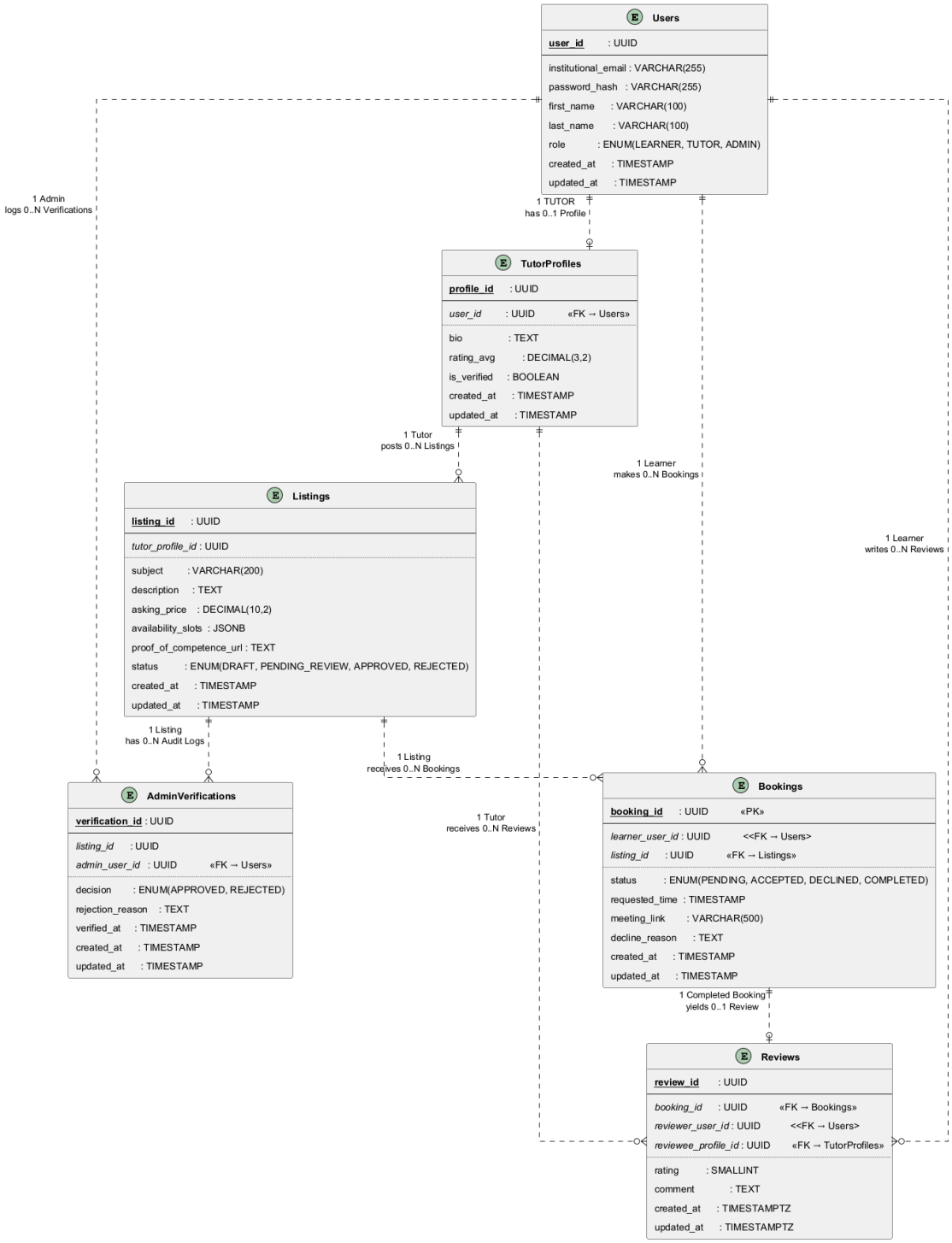
- **BOOKING-001:** Scheduling conflict (time slot is no longer available)
- **BOOKING-002:** Missing required workflow data (e.g., no meeting link or rejection reason provided)
- **LISTING-001:** Action prohibited on unverified listing (e.g., Learner trying to book a Tutor pending Admin approval)
- **LISTING-002:** Invalid pricing (e.g., Asking price exceeds campus guidelines)

System & Infrastructure

- **SYSTEM-001:** Internal server error
- **SYSTEM-002:** Cloud storage integration failure (e.g., unable to fetch or upload Proof of Competence)

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **One-to-One: User ↔ TutorProfile**
  - (A user who signs up as a tutor has exactly one detailed profile; learners do not have a tutor profile.)
- **One-to-One: Booking ↔ Review**
  - (Each completed tutoring session can result in exactly one official review/rating.)
- **One-to-Many: TutorProfile → Listings**
  - (A single tutor can create and manage multiple specialized subject listings.)
- **One-to-Many: User (Learner) → Bookings**
  - (A learner can initiate multiple booking requests across different tutors.)
- **One-to-Many: User (Admin) → AdminVerifications**
  - (An administrator logs multiple verification actions for different tutor applications.)
- **One-to-Many: Listing → Bookings**
  - (One specific subject listing can receive multiple booking requests from various learners.)
- **One-to-Many: Listing → AdminVerifications**
  - (A listing can have multiple audit logs if it was rejected, edited, and then re-verified.)
- **Many-to-One: Bookings → Listing**
  - (Multiple booking records reference a single parent listing for subject and price data.)
- **Many-to-One: Reviews → TutorProfile**
  - (Multiple reviews from different learners are aggregated under one tutor's profile to calculate their average rating.)

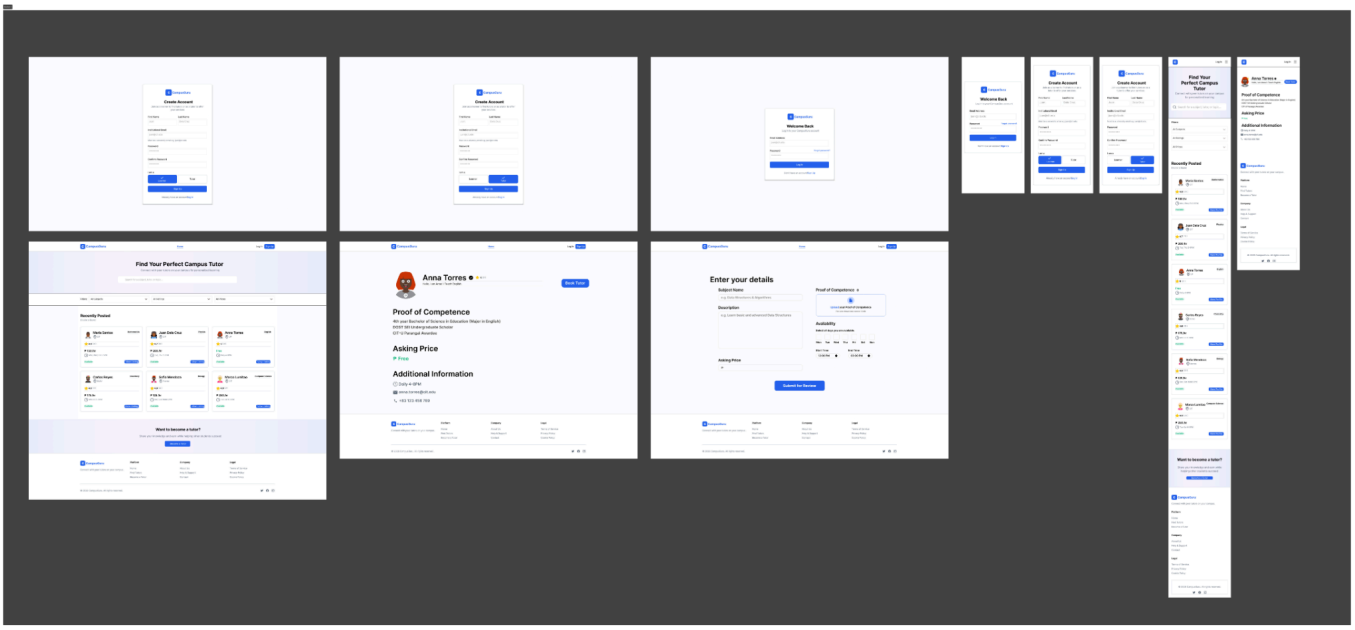
Key Tables:

1. **users -** Core accounts and authentication (University domain restricted).
2. **tutor_profiles -** Extended profiles for students acting as tutors.
3. **listings -** Subject-specific tutor offerings and competence proof.
4. **bookings -** Transactional records of session requests and status.
5. **admin_verifications -** Audit logs for listing approvals and rejections.
6. **reviews -** Learner feedback and ratings for completed sessions.

Table Structure Summary:

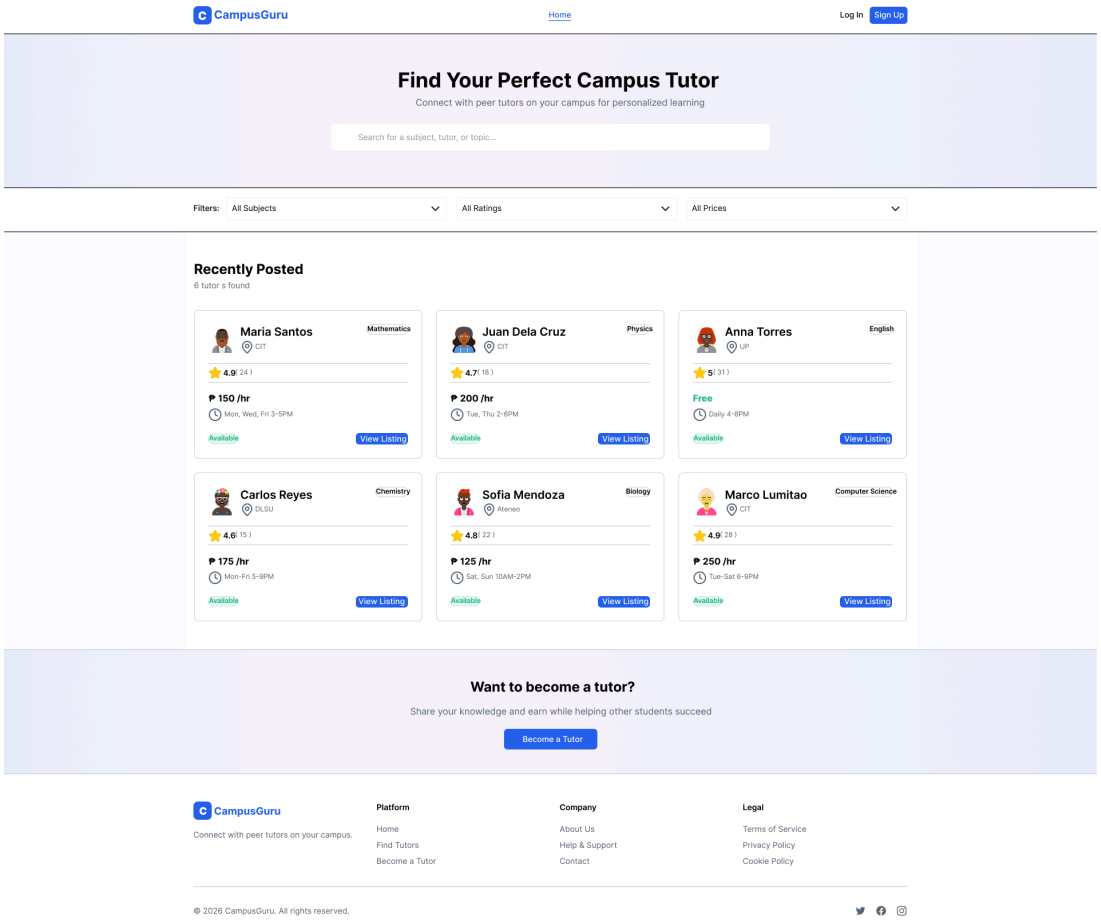
- **users:** user_id, institutional_email, password_hash, first_name, last_name, role, created_at, updated_at
- **tutor_profiles:** profile_id, user_id, bio, average_rating, is_verified, profile_picture_url
- **listings:** listing_id, tutor_profile_id, subject, description, asking_price, availability_slots, proof_of_competence_url, status
- **bookings:** booking_id, learner_id, listing_id, requested_time, status, meeting_link, decline_reason, created_at
- **admin_verifications:** verification_id, admin_id, listing_id, action (Approve/Reject), rejection_reason, verified_at
- **reviews:** review_id, booking_id, learner_id, tutor_profile_id, rating (1-5), comment, created_at

7.0 UI/UX DESIGN



7.1 Web Application Prototype

Homepage (Dashboard)



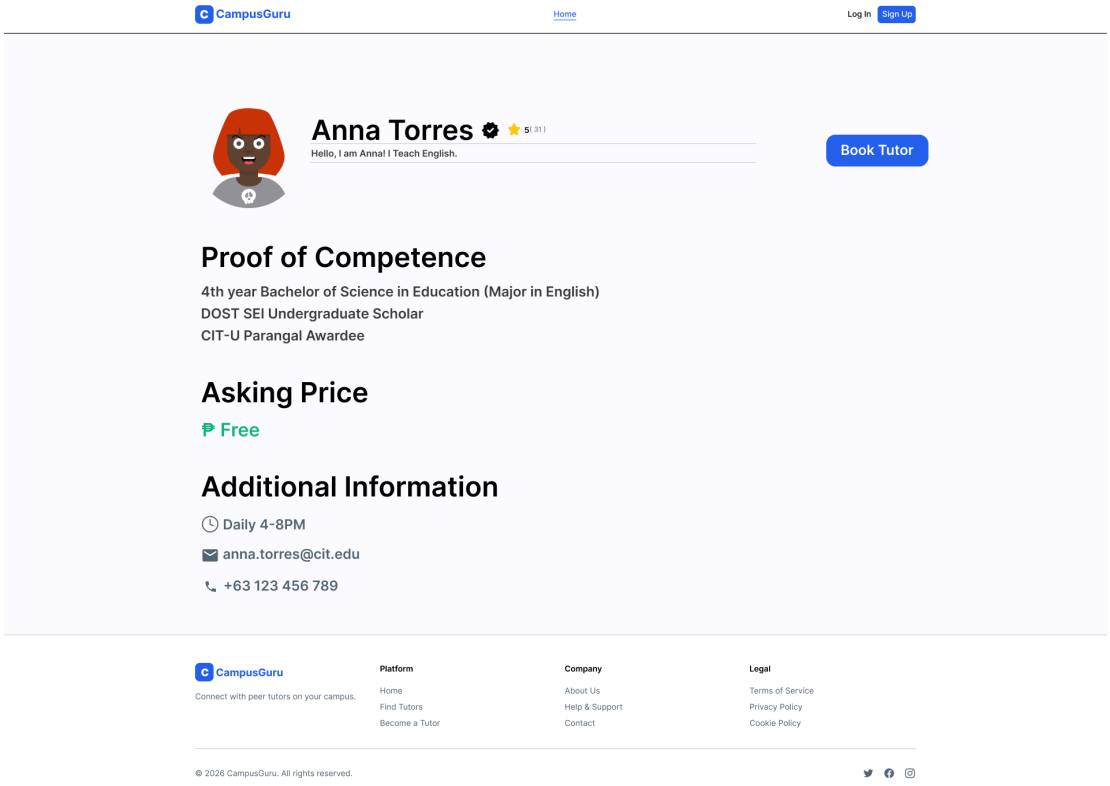
Header: [Logo] [Home] ([Log in] [Sign up] / Profile Picture) [Search Bar] [Filters]

Content: Recently Posted Listings (Grid Layout)

Each Card: Profile Picture, Name, Asking Price, Available Time, Subject, "View Listing" button

Footer: Links, Copyright

View Listing Page



Profile Picture of Tutor

Tutor Name, Verification Badge, Rating Average

Tutor's Bio

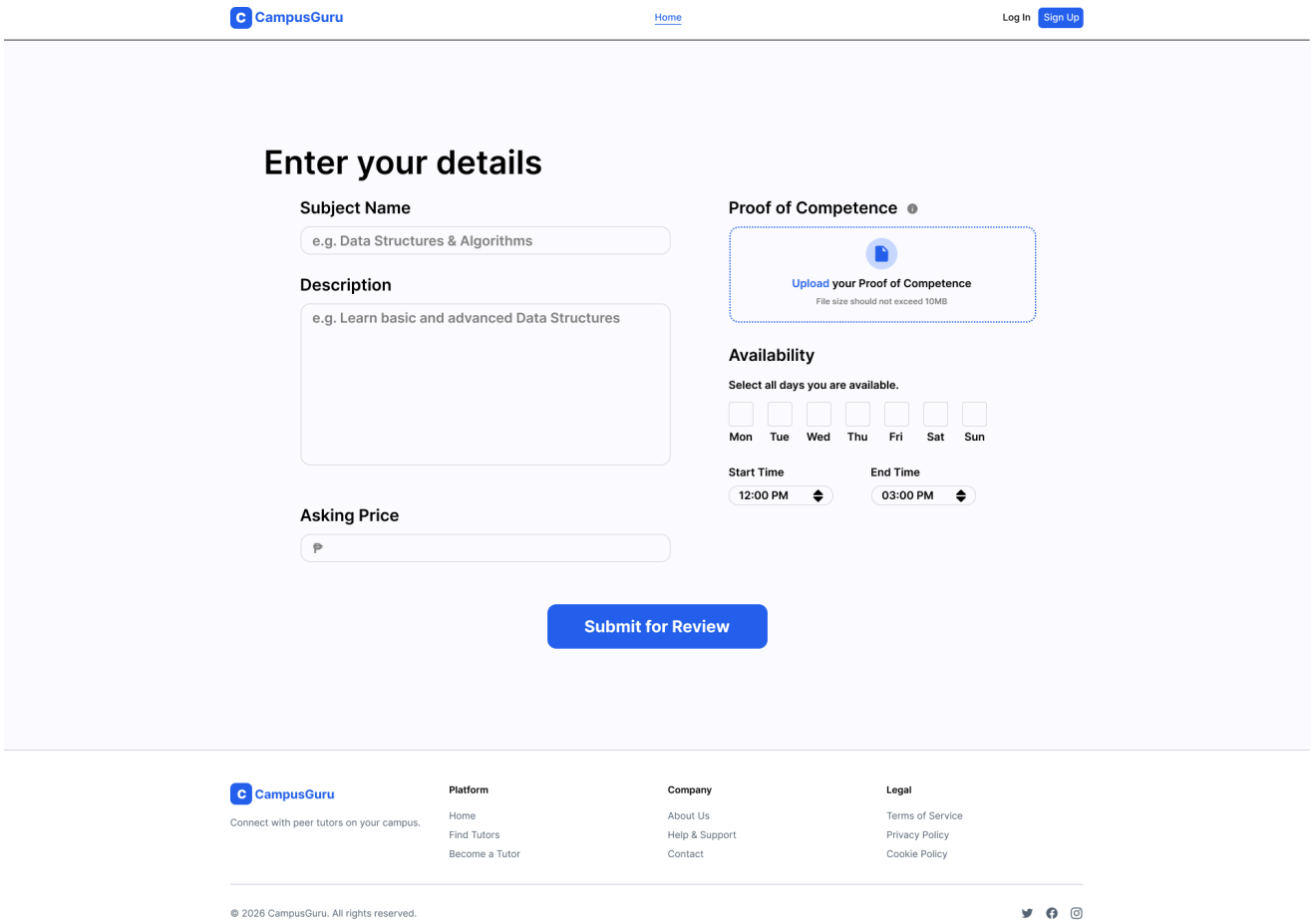
Public Proof of Competence / Academic Achievements

Asking Price

Additional Information (Available Time, Email, Contact Information)

Book Tutor

Apply for Listing as a Tutor



“Enter your details” prompt

Input Subject name

Input Description

Input Asking Price

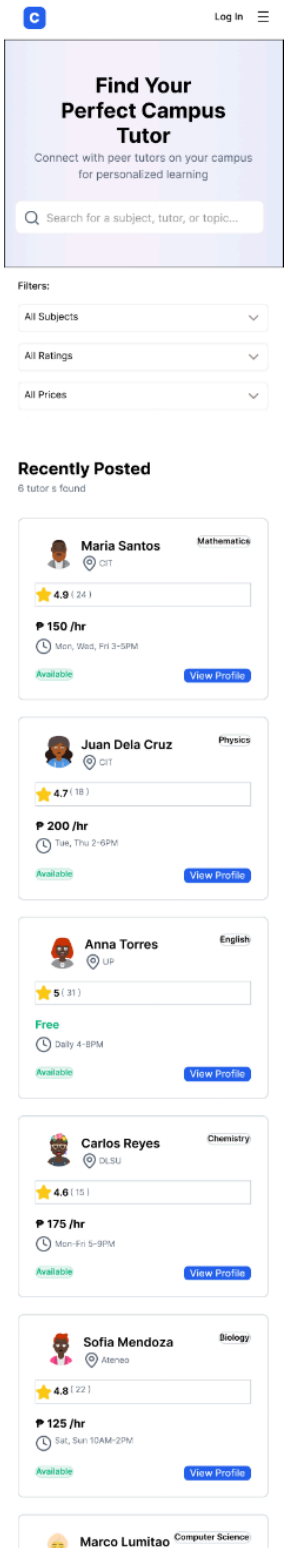
Upload Proof of Competence (grades, certificates, etc.)

Availability (Days within the week and time range)

Submit button

7.2 Mobile Application Prototype

Homepage (Dashboard)



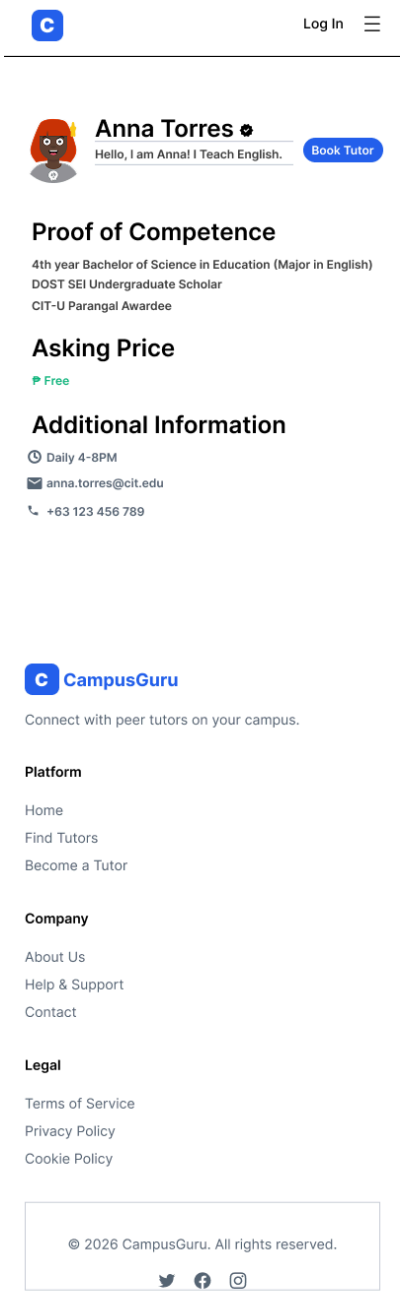
Header: [Logo] ([Log in]] / Profile Picture) [Burger Menu] [Search Bar] [Filters]

Content: Recently Posted Listings (Grid Layout)

Each Card: Profile Picture, Name, Asking Price, Available Time, Subject, "View Listing" button

Footer: Links, Copyright

View Listing Page



Profile Picture of Tutor

Tutor Name, Verification Badge, Rating Average

Tutor's Bio

Public Proof of Competence / Academic Achievements

Asking Price

Additional Information (Available Time, Email, Contact Information)

Book Tutor

Mobile-Specific Features:

- Touch-optimized buttons (min 44x44px)
- Gesture support (swipe, pull-to-refresh)
- Simplified forms for mobile input (such as the header)

Design System:

- **Colors:**
  - Primary (#2463EB)
  - Secondary (#EEF0FD)
  - Success (#10B77F)
  - Error (#B71013)
- **Typography:**
  - Inter font family, responsive sizing
- **Spacing:**
  - 8px grid system
- **Components:**
  - Consistent buttons, inputs, cards, modals
- **Responsive:**
  - Mobile-first approach, breakpoints at 640px, 768px, 1024px

8.1 Project Timeline

Phase 1: Planning & Design (Weeks 1-2)

- **Week 1: Requirements & Architecture**
  - Day 1-2: Project setup, Git repository initialization, and documentation.
  - Day 3-4: Finalize FRS (Tutor/Learner flows) and NFRs (Sync & Security).
  - Day 5-7: **System Architecture Design:** Finalize the 3-tier structure (React, Kotlin, Spring Boot).
- **Week 2: Detailed Design**
  - Day 1-2: **API Specification:** Complete JWT and Marketplace endpoint definitions.
  - Day 3-4: **Database Design:** MySQL ERD and schema script creation.
  - Day 5-6: **UI/UX Wireframes:** Draft the "View-Only" Admin modal and Tutor Dashboard.
  - Day 7: Implementation plan finalization and project kickoff.

Phase 2: Backend Development (Weeks 3-4)

- **Week 3: Foundation & Security**
  - Day 1: Spring Boot initialization with MySQL and JPA dependencies.
  - Day 2: **Entity Implementation:** Mapping Users, TutorProfiles, and Listings.
  - Day 3: **Campus-Exclusive Auth:** JWT implementation with institutional email validation.
  - Day 4: Auth endpoints (Register/Login) with role-based access control.
  - Day 5: **Media Integration:** Setup Supabase/S3 SDK for "Proof of Competence" uploads.
- **Week 4: Core Marketplace Logic**
  - Day 1: **Listing CRUD:** Tutors can create, update, and delete service listings.
  - Day 2: **Booking Engine:** Implementing the "Pending $\rightarrow$ Accepted" state machine.

    - Day 3: Search and Filtering logic (by subject, price, and rating).
    - Day 4: **Admin Tools:** Verification endpoints and moderation logic.
    - Day 5: **JMeter Load Testing:** Verify the 30 concurrent user NFR; finalize API documentation.

Phase 3: Web Portal Development (Weeks 5-6)

- **Week 5: Student & Tutor Web Interface**
    - Day 1: React setup with Tailwind CSS and Axios.
    - Day 2: Authentication pages and University-domain validation UI.
    - Day 3: **Listing Feed:** Home page for browsing and filtering tutors.
    - Day 4: **Tutor Dashboard:** Table for "Pending Requests" and Availability management.
    - Day 5: **Booking Flow:** Modal for link-pasting and rejection reasons.
- **Week 6: Admin Portal & Feedback**
    - Day 1: **Admin Dashboard:** Credential verification view with the "View-Only" PoC modal.
    - Day 2: Moderation tools (Hiding listings/Deleting reviews).
    - Day 3: **Rating & Review System:** Post-session feedback components.
    - Day 4: Responsive UI polish for desktop and mobile browsers.
    - Day 5: Integration testing with the live Spring Boot API.

Phase 4: Android Mobile Application (Weeks 7-8)

- **Week 7: Mobile Foundation**
    - Day 1: Android Studio (Kotlin) setup and Retrofit API client configuration.
    - Day 2: **Mobile Auth:** Login/Register screens with persistent JWT storage.
    - Day 3: **Tutor Browsing:** Recycler View for Listings with Subject filtering.
    - Day 4: Tutor Profile screen with "Proof of Competence" display.
    - Day 5: **Booking Request:** Mobile form for scheduling sessions.
- **Week 8: Mobile Workflow & Polish**
    - Day 1: **Notification/Status View:** Check booking status (Accepted/Declined).

    - Day 2: **Session Completion:** Rating and reviewing tutors via mobile UI.
    - Day 3: UI/UX polish (Material Design 3) and animations.
    - Day 4: Testing on multiple Android emulators and physical devices.
    - Day 5: APK generation and mobile-specific technical documentation.

Phase 5: Final Integration & Deployment (Weeks 9-10)

- **Week 9: System Integration Testing (SIT)**
    - Day 1: **Cross-Platform Sync:** Ensure actions on Android reflect on Web and vice-versa.
    - Day 2: End-to-end testing of the "Request $\rightarrow$ Approve $\rightarrow$ Rate" lifecycle.
    - Day 3: **Security Audit:** Pentesting for SQL Injection and JWT unauthorized access.
    - Day 4: Final bug fixes and UI/UX edge-case handling.
    - Day 5: Update the final project documentation.

- **Week 10: Production Deployment**
    - Day 1: **Backend Deployment:** Hosting Spring Boot (Railway/Render) and MySQL.
    - Day 2: **Web Deployment:** Hosting the React portal (Vercel/Netlify).
    - Day 3: Mobile APK distribution and environment variable finalization.
    - Day 4: Final system "Sanity Test" in the production environment.
    - Day 5: **Project Submission and Presentation.**

8.2 Project Milestones

- **M1 (End Week 2):** Design finalized (ERD, API Contracts, Component Diagrams).
- **M2 (End Week 4):** Backend API is feature-complete and load-tested.
- **M3 (End Week 6):** Web Portal (Student/Admin) fully integrated with the API.
- **M4 (End Week 8):** Android App fully functional and synced with the database.
- **M5 (End Week 10):** Production system live and integrated across all tiers.

8.3 Risk Mitigation

1. **Scope Control:** Stick to the MoSCoW "Must-Have" features first (Auth, Listings, Bookings).
2. **Early Integration:** Connect the React and Kotlin apps to the API in Week 5/7 to catch data format mismatches early.
3. **Data Persistence:** Keep daily database backups during development to avoid losing test data.
4. **Testing Strategy:** Use JMeter for early performance testing to ensure the "Systems Integration" requirement is met.